

Lecture Unit 7.1

Parallel scaling and parallel fraction

Performance degradation

SAB: section 1.4.5

- A number of factors can contribute to real-world parallel performance being less than the idealised peak performance:
 - **S**tarvation: insufficient parallel work to keep the processors busy (**insufficient parallel work**, uneven distribution of work)
 - **L**atency: time taken for information to travel from one part of the system to another (e.g. accessing memory, sending messages)
 - **O**verhead: work required in addition to the computation (e.g. starting and stopping OpenMP parallel regions)
 - **W**aiting: multiple threads/processes accessing a shared resource (e.g. contention for memory or network bandwidth)

Unit overview

- Definitions of parallel speed-up and parallel efficiency
- Strong scaling and Amdahl's Law Weak scaling and Gustafson's Law
- The workshop uses information from units 7.1 and 7.2

Parallel speed-up

- How much faster is the parallel program than the serial version? This is called the parallel speed-up:

$$S_N = \frac{T_0}{T_N}$$

where T_0 is the time for the serial program to run and T_N is the time taken for the parallel program to run on N processors

- For example if we halve the run time by running on N processors then the speed-up is 2

Parallel efficiency

- Does this speed-up represent efficient use of the resources?
- We can define a parallel efficiency:

$$E_N = \frac{S_N}{N}$$

where S_N is the speed-up on N processors

- N times faster on N processors is an efficiency of 1

Strong scaling

SAB: section 1.4.5

- Extra performance lets us do the same calculation quicker
- In strong scaling we keep the **total** problem size fixed and increase number of processors to reduce the run time
- For example parallelising a 1024x1024 domain using domain decomposition:
 - 4 processors: 2x2 sub-domains with 512x512 points
 - 16 processors: 4x4 sub-domains with 256x256 points
 - 64 processors: 8x8 sub-domains with 128x128 points

Amdahl's law

SAB 2.7.2

- How much faster can we calculate a solution to a given problem by parallelising the calculation?
- The parallel speed-up is limited by the fraction of the program which can be parallelised (Amdahl, 1967)
- We can use this to work out the speed-up under the assumption that the only limiting factor is the parallel fraction

Gene M. Amdahl. 1967. Validity of the single processor approach to achieving large scale computing capabilities. In Proceedings of the April 18-20, 1967, spring joint computer conference (AFIPS '67 (Spring)). Association for Computing Machinery, New York, NY, USA, 483–485. DOI:<https://doi.org/10.1145/1465482.1465560>

Amdahl's law

- Consider a program where a fraction p of the workload can be parallelised and a fraction $s = 1 - p$ cannot be parallelised. The serial run time is given by:

$$T_0 = sT_0 + pT_0$$

- If we can parallelise a fraction p of the workload with perfect efficiency the run time on N processors (T_N) is given by:

$$T_N = sT_0 + \frac{pT_0}{N}$$

- The parallel speed-up (S_N) is given by:

$$S_N = \frac{T_0}{T_N} = \frac{sT_0 + pT_0}{sT_0 + \frac{pT_0}{N}} = \frac{s + p}{s + \frac{p}{N}} = \frac{1}{s + \frac{p}{N}}$$

Maximum speed-up

$$S_N = \frac{1}{s + \frac{p}{N}} \text{ is known as Amdahl's law}$$

- According to Amdahl's law the parallel fraction of a program limits the maximum possible speed-up
- If we could run on infinitely many processors with perfect efficiency the maximum speed-up would be:

$$S_{\max} = \frac{1}{s} = \frac{1}{1 - p}$$

Speed up vs. number of processors and parallel fraction

	p=0.5	p=0.9	p=0.99	p=0.999
N=10	1.82	5.26	9.17	9.91
N=100	1.98	9.17	50.25	90.99
N=1000	2.00	9.91	90.99	500.25
N=500 000	2.00	10.00	99.98	998.00
Maximum speed-up	2	10	100	1000

The parallel fraction must be very high to make effective use of a large number of processors.

“Re-evaluating Amdahl’s law”

- Amdahl’s law is bad news for parallel processing
- “When given a more powerful processor the problem generally expands to make use of the increased facilities” (Gustafson, 1988)
- Gustafson observed that “in practice, the problem size scales with the number of processors”

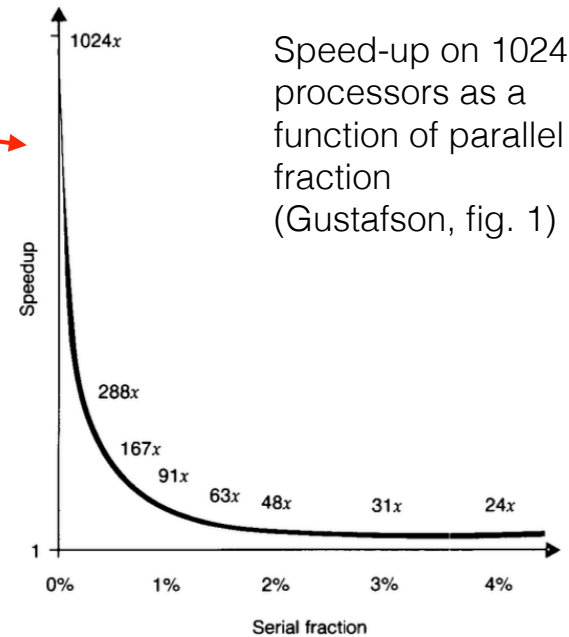


FIGURE 1. Speedup under Amdahl's Law

John L. Gustafson. 1988. Reevaluating Amdahl's law. Commun. ACM 31, 5 (May 1988), 532–533.
DOI:<https://doi.org/10.1145/42411.42415>

Gustafson's law

- Gustafson considered a serial part which does not grow with the problem size and a parallel part which scales linearly with the number of processors

A serial processor would run this calculation in time:

$$T_0 = s + pN$$

An ideal parallel computer would run this in time:

$$T_N = s + \frac{pN}{N} = s + p = 1$$

The speed-up is:

$$S_N = \frac{T_0}{T_N} = s + pN$$

Weak scaling

- $S_N = s + pN$ is known as Gustafson's law
- The speed-up increases linearly with N , so the run time remains constant, and it is easier to make use of a large number of processors
- Gustafson's law describes **weak scaling**
- In weak scaling the problem size **per processor** remains fixed
- With a problem size of 128x128 per processor main using domain decomposition:
 - 4 processors: 256x256 total size
 - 16 processors: 512x512 total size
 - 64 processors: 1024x1024 total size

Unit summary

- Definitions of parallel speed up and parallel efficiency
- Strong scaling:
 - Total problem size remains fixed
 - Described by Amdahl's law
- Weak scaling:
 - Problem size per processor remains fixed Described by Gustafson's law
 - processor main using domain decomposition: